

FACTORIZED DELAYED STREAMS MODELING FOR LLM-BASED STREAMING ASR

Tatsunari Takagi^{1,2}, Kai Washizaki¹, Atsushi Kojima¹, Lianbo Liu¹, Koki Nikaido¹, Yui Sudo¹

¹SB Intuitions

²Toyohashi University of Technology

ABSTRACT

Delayed Streams Modeling (DSM) enables LLM-based streaming automatic speech recognition (ASR) by aligning acoustic and text streams on a common timeline. DSM adds the padding token $\langle p \rangle$ and the word-start token $\langle w \rangle$ to the LLM vocabulary and predicts them together with normal text tokens using the same softmax. We first show that $\langle w \rangle$ can be removed while maintaining competitive recognition performance. Based on this result, we propose Factorized DSM (F-DSM), which separates the waiting probability for $\langle p \rangle$ from the distribution over the original LLM vocabulary. This factorization removes ASR-specific tokens from the text prediction space and allows the large-vocabulary softmax to be skipped on waiting steps. Experiments on the Corpus of Spontaneous Japanese and LibriSpeech show that F-DSM achieves better recognition performance than DSM. It also greatly reduces GPU memory use while maintaining similar training throughput, provides a small inference speed improvement through softmax skipping, and reduces the degradation in text-only perplexity observed with DSM.

Index Terms— Delayed Streams Modeling, streaming automatic speech recognition, large language models, Hybrid Autoregressive Transducer

1. INTRODUCTION

Streaming automatic speech recognition (ASR) processes incoming speech step by step and produces text as soon as enough acoustic information is available [1, 2]. At each step, the model must decide whether to output a text token or wait for more speech. In Connectionist Temporal Classification (CTC) [3] and Recurrent Neural Network-Transducer (RNN-T) [4], this “no output” state is modeled by a blank symbol, which plays a different role from normal text tokens.

Recently, large language models (LLMs) [5, 6] have been widely studied for ASR because of their strong language modeling ability [7–9]. However, standard LLMs are designed to generate text autoregressively and do not have a natural mechanism to control when a token should be emitted while speech is continuously arriving.

Delayed Streams Modeling (DSM) addresses this problem by using the same decoder-only LLM to process time-aligned acoustic and text streams [10, 11]. In addition to normal text tokens, DSM adds two special tokens to the LLM vocabulary. The padding token $\langle p \rangle$ means that the model should wait without producing text, while the word-start token $\langle w \rangle$ marks the beginning of a word. These special tokens are predicted together with normal text tokens using the same softmax.

This simple design allows an LLM to perform streaming ASR, but it also places ASR-specific timing tokens and normal

text tokens in the same output distribution. In a time-aligned sequence, many steps have no text output and therefore use $\langle p \rangle$ as the target. As a result, $\langle p \rangle$ appears much more often than any individual text token. Still, DSM computes the projection and softmax over the full vocabulary even when the model only needs to wait. In addition, $\langle p \rangle$ and $\langle w \rangle$ are included in the softmax normalization, so these ASR-specific tokens compete with the distribution over the original LLM vocabulary learned during pretraining. Streaming ASR therefore involves two distinct decisions: whether to emit a token and, if so, which token to emit. DSM couples these decisions in a single softmax, motivating us to model timing and lexical prediction separately.

We first study whether $\langle w \rangle$ is necessary for speech recognition. Our experiments show that removing $\langle w \rangle$ still gives competitive recognition performance. This result suggests that the ASR-specific output space of DSM can be simplified to only one special token, $\langle p \rangle$. The remaining $\langle p \rangle$ can be viewed as a timing symbol rather than a normal text token, similar to the blank symbol in CTC and RNN-T. Based on this view, we propose Factorized DSM (F-DSM), inspired by the Hybrid Autoregressive Transducer (HAT) [12]. F-DSM separates the waiting probability for $\langle p \rangle$ from the distribution over the original LLM vocabulary. With this factorization, the text prediction head keeps the same output space as the pretrained LLM and does not include any ASR-specific token. In addition, when the model decides to wait, it does not need to compute the large-vocabulary projection and softmax. F-DSM therefore separates ASR-specific timing prediction from the original text-token prediction of the LLM while keeping the streaming framework of DSM.

We evaluate F-DSM on Japanese and English using the Corpus of Spontaneous Japanese (CSJ) [13] and LibriSpeech [14]. F-DSM achieves better recognition performance than the original DSM. During training, it greatly reduces GPU memory use while keeping similar training throughput, resulting in more memory-efficient training. During inference, skipping the large-vocabulary softmax on waiting steps provides a small speed improvement. We also evaluate text-only language modeling. Compared with the original DSM, F-DSM reduces the degradation in perplexity (PPL) of the pretrained LLM, showing that separating ASR-specific tokens from the original LLM vocabulary better preserves its language modeling behavior.

These results show that treating $\langle p \rangle$ separately from normal text tokens is useful not only for reducing computation but also for preserving the original language modeling ability of the LLM. The factorized design requires only a small change to DSM while keeping its streaming framework unchanged. This makes F-DSM a simple and effective approach for LLM-based streaming ASR.

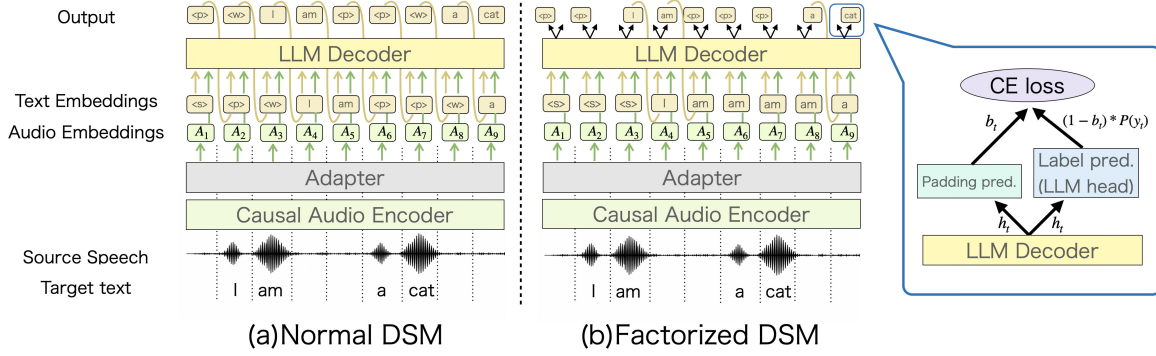


Fig. 1. DSM (left) jointly predicts waiting $\langle p \rangle$, word-start $\langle w \rangle$, and text tokens; F-DSM (right) separates waiting from text prediction over the original LLM vocabulary.

2. PRELIMINARIES

2.1. Delayed Streams Modeling

DSM [10] is a method for streaming speech recognition with a decoder-only LLM. As shown on the left side of Fig. 1, DSM aligns acoustic and text streams in time and shifts the text stream by a fixed delay. This allows the model to predict the next token sequentially using acoustic information and previous text.

In addition to normal text tokens, DSM adds two special tokens to the LLM vocabulary. The padding token $\langle p \rangle$ represents waiting without producing text, while the word-start token $\langle w \rangle$ marks the beginning of a word. Let $\mathcal{V}$ denote the original LLM vocabulary. The output vocabulary of DSM is defined as

$$\mathcal{V}_{\text{DSM}} = \mathcal{V} \cup \{\langle p \rangle, \langle w \rangle\}. \quad (1)$$

Let t index aligned time frames, y_t be the output token, and $\mathbf{h}_t \in \mathbb{R}^d$ be the decoder hidden state. DSM predicts each token $v \in \mathcal{V}_{\text{DSM}}$ as

$$P_{\text{DSM}}(y_t = v \mid \mathbf{h}_t) = \text{Softmax}(\mathbf{W}_{\text{DSM}} \mathbf{h}_t)_v, \quad (2)$$

where $\mathbf{W}_{\text{DSM}} \in \mathbb{R}^{(|\mathcal{V}|+2) \times d}$. This full-vocabulary projection is computed at every frame, including waiting frames.

The word-start token $\langle w \rangle$ was introduced to explicitly mark the transition from padding to text and support timestamp extraction during inference [10, 11]. However, its necessity has not been established through an ablation that removes it. ASR systems can provide word-level timestamps without a dedicated word-start token [15], suggesting that $\langle w \rangle$ may also be unnecessary in DSM. This motivates our ablation of $\langle w \rangle$ for recognition accuracy.

2.2. Hybrid Autoregressive Transducer

HAT [12] factorizes the output distribution of RNN-T into a blank probability and a conditional distribution over non-blank labels [16, 17]. This separates the decision of whether to emit a label from the decision of which label to emit.

At acoustic frame t after u emitted labels, let $\mathbf{h}_{t,u}$ be HAT's joint hidden state, $b_{t,u}$ its blank probability, and $q_{t,u}(v)$ its conditional distribution over non-blank labels $v \in \mathcal{V}$. Then

$$P_{\text{HAT}}(v \mid \mathbf{h}_{t,u}) = \begin{cases} b_{t,u}, & v = \langle \text{blank} \rangle, \\ (1 - b_{t,u})q_{t,u}(v), & v \in \mathcal{V}. \end{cases} \quad (3)$$

This factorization motivates treating the waiting token $\langle p \rangle$ separately from text tokens in DSM.

3. FACTORIZED DELAYED STREAMS MODELING

3.1. Overview

Building on the DSM framework and the HAT factorization described in Sec. 2, we propose F-DSM, which removes $\langle w \rangle$ and separates the waiting probability for $\langle p \rangle$ from the distribution over the original LLM vocabulary.

Following HAT and RNN-T, we feed back only emitted text tokens to the LLM decoder for autoregressive prediction, excluding the waiting token $\langle p \rangle$.

3.2. Factorized Output Distribution

Let $\mathbf{h}_t \in \mathbb{R}^d$ denote the LLM decoder hidden state at time frame t , as in DSM. F-DSM first computes the waiting probability b_t using the padding prediction head as

$$b_t = \sigma(\mathbf{w}_p^\top \mathbf{h}_t + c_p), \quad (4)$$

where σ is the sigmoid function, $\mathbf{w}_p \in \mathbb{R}^d$ is a learned weight vector, and $c_p \in \mathbb{R}$ is a bias. Thus, the padding head is a scalar linear layer followed by a sigmoid.

For normal text tokens, the original LLM head predicts a distribution over the original LLM vocabulary $\mathcal{V}$ as

$$P_{\text{LLM}}(v \mid \mathbf{h}_t) = \text{Softmax}(\mathbf{W}_{\text{LLM}} \mathbf{h}_t)_v, \quad v \in \mathcal{V}. \quad (5)$$

Here, $\mathbf{W}_{\text{LLM}} \in \mathbb{R}^{|\mathcal{V}| \times d}$ is the pretrained LLM output matrix. Its projection costs $O(|\mathcal{V}|d)$ operations per frame, versus $O(d)$ for the padding head. Unlike DSM in Eq. 2, F-DSM can skip this expensive projection at waiting positions.

In contrast to the joint softmax in Eq. 2, F-DSM defines its output probability as

$$P_{\text{F-DSM}}(y_t \mid \mathbf{h}_t) = \begin{cases} b_t, & y_t = \langle p \rangle, \\ (1 - b_t)P_{\text{LLM}}(y_t \mid \mathbf{h}_t), & y_t \in \mathcal{V}. \end{cases} \quad (6)$$

This follows the HAT factorization in Eq. 3, with $\langle p \rangle$ serving as the waiting symbol. In this formulation, $\langle p \rangle$ is not included in the original LLM vocabulary but is modeled separately as the waiting probability. Therefore, the LLM head keeps the same output space as in pretraining.

3.3. Training and Streaming Inference

F-DSM is trained using cross-entropy loss with the factorized output probability defined above. The loss at step t is written as

$$\mathcal{L}_t = \begin{cases} -\log b_t, & y_t = \langle p \rangle, \\ -\log(1 - b_t) - \log P_{\text{LLM}}(y_t | \mathbf{h}_t), & y_t \in \mathcal{V}. \end{cases} \quad (7)$$

For $\langle p \rangle$ targets, the vocabulary projection and softmax in Eq. 5 can be skipped during training, reducing memory use for large-vocabulary logits.

During streaming inference, the padding prediction head first computes b_t . Similar to blank-thresholding approaches for HAT-based ASR [17, 18], F-DSM uses the waiting probability to avoid expensive non-waiting computation. Specifically, under greedy decoding, F-DSM skips the LLM vocabulary projection and softmax when the waiting probability exceeds 0.5:

$$b_t > 0.5 \implies \text{skip Eq. 5.} \quad (8)$$

At these steps, the model outputs $\langle p \rangle$ and moves to the next speech step.

4. EXPERIMENTS

4.1. Experimental Setup

We use CSJ for Japanese experiments and LibriSpeech for English experiments. The base LLM is Qwen3.5-0.8B [19], and the acoustic encoder is the Fast-Conformer [20] encoder from Nemotron-3.5-ASR-Streaming-0.6B [2, 21]. The encoder operates at a frame rate of 12.5 Hz. A randomly initialized two-layer MLP is used as an audio projection adapter between the acoustic encoder and the LLM.

The speech and text in both datasets are aligned using Qwen3-Aligner [9], and the alignments are represented at the 12.5-Hz frame rate of the acoustic encoder. During training, the target delay is uniformly sampled from 160, 320, 480, 640, and 800 ms. Following Voxtral Realtime, we use Ada RMSNorm to condition the model on the target delay [11].

We compare DSM w/ $\langle w \rangle$, DSM w/o $\langle w \rangle$, and F-DSM. The second removes only $\langle w \rangle$ from DSM, while F-DSM further factorizes the prediction of $\langle p \rangle$ as described in Sec. 3.

For the two DSM baselines, we apply z-loss regularization, following the setting reported to be effective in Voxtral Realtime [11]. F-DSM does not use z-loss regularization because $\langle p \rangle$ is not included in the LLM vocabulary.

Training is performed on four NVIDIA A100 GPUs with a global batch size of 128. Each model is trained for a total of 100,000 steps using a three-stage training schedule. For the first 5,000 steps, only the audio projection adapter and the newly added prediction parameters are updated. For the next 5,000 steps, the acoustic encoder is additionally updated. For the remaining 90,000 steps, the acoustic encoder, adapter, prediction parameters, and LoRA [22] parameters are updated. The base LLM parameters remain frozen, and the LLM is adapted using LoRA.

We use AdamW [23] with zero weight decay. For all models, we use the learning-rate setting that gives the best performance for DSM w/ $\langle w \rangle$. The learning rates are 3×10^{-4} for the language-model adapters, 10^{-5} for the acoustic encoder, 10^{-4} for the audio projection and delay-conditioning modules, and 5×10^{-5} for the streaming-token parameters.

Table 1. ASR error with greedy decoding, 480 ms target delay, and no left padding (lower is better).

Model	CSJ (CER, %)				LibriSpeech (WER, %)		
	eval1	eval2	eval3	Ave.	test-clean	test-other	Ave.
DSM w/ $\langle w \rangle$	7.91	6.80	7.08	7.27	3.86	9.17	6.52
DSM w/o $\langle w \rangle$	7.78	6.46	5.54	6.60	3.83	9.38	6.61
F-DSM	7.54	6.29	5.50	6.45	3.66	8.68	6.17

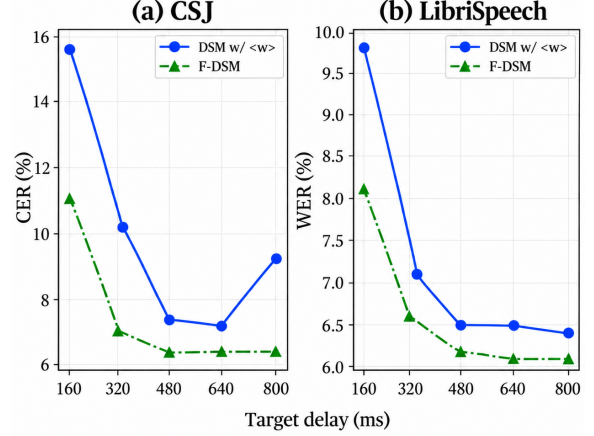


Fig. 2. Error versus target delay: (a) CSJ CER and (b) LibriSpeech WER (lower is better).

Recognition performance is evaluated using greedy decoding, with LLM-head skipping according to Eq. 8 for F-DSM. We use character error rate (CER) for CSJ and word error rate (WER) for LibriSpeech.

4.2. Speech Recognition Accuracy

Table 1 compares the three models at a target delay of 480 ms. Removing $\langle w \rangle$ improves CER on CSJ while changing WER only slightly on LibriSpeech, indicating that explicit word-start prediction is not required for competitive recognition in our setting. F-DSM further improves the average CER from 6.60% to 6.45% on CSJ and the average WER from 6.61% to 6.17% on LibriSpeech.

Fig. 2 compares F-DSM with DSM w/ $\langle w \rangle$ at target delays of 160–800 ms. F-DSM achieves lower error rates at every evaluated delay on both datasets, with the largest gap at 160 ms. For F-DSM, error rates decrease as the delay increases to 480 ms, with little further change at longer delays. On CSJ, DSM’s CER rises at 800 ms, whereas F-DSM’s CER remains stable. These results show that F-DSM improves recognition across the evaluated delay settings, including the shortest target delay.

4.3. Training Efficiency

Table 2 compares training time and GPU memory use. As discussed in Sec. 3, in F-DSM, the large LLM head does not need to be evaluated at positions where the target is $\langle p \rangle$, reducing the memory required for large-vocabulary logits.

Table 2. Training efficiency at batch sizes 16 / 32 / 64. OOM means out of memory. — means VRAM unavailable due to OOM.

Model	Time (s/step) ↓	Ave. VRAM (GiB) ↓
<i>CSJ (mean utterance: 4.63 s)</i>		
DSM w/ <w>	0.886 / 1.309 / OOM	35.14 / 61.30 / —
DSM w/o <w>	1.026 / 1.427 / OOM	35.09 / 60.95 / —
F-DSM	1.043 / 1.375 / 2.478	14.71 / 18.49 / 25.58
<i>LibriSpeech (mean utterance: 12.30 s)</i>		
DSM w/ <w>	1.178 / 1.771 / OOM	38.34 / 65.82 / —
DSM w/o <w>	1.133 / 1.771 / OOM	37.49 / 64.66 / —
F-DSM	1.100 / 1.714 / 3.008	18.04 / 24.51 / 38.69

Table 3. Streaming inference efficiency (CSJ / LibriSpeech). Skip rate: decoding steps skipping vocabulary projection and softmax (%). RTF: processing time/audio duration. —: not applicable.

Model	Softmax skip rate (%)	RTF ↓
DSM w/ <w>	— / —	0.827 / 0.780
DSM w/o <w>	— / —	0.824 / 0.764
F-DSM	74.70 / 66.89	0.811 / 0.759

At batch size 32, DSM w/o <w> uses 60.95 GiB on CSJ, while F-DSM uses only 18.49 GiB. On LibriSpeech, GPU memory use is reduced from 64.66 GiB to 24.51 GiB. At the same time, the training time per step remains similar: 1.427 s versus 1.375 s on CSJ and 1.771 s versus 1.714 s on LibriSpeech.

On CSJ, whose mean utterance length is 4.63 s, DSM w/ <w> is faster, likely because it avoids the overhead of selectively skipping vocabulary computation. For the longer utterances in LibriSpeech (12.30 s on average), this overhead is less significant relative to the total computation, and F-DSM is slightly faster at matched batch sizes. We expect longer utterances to further reduce the relative impact of this overhead and make the memory savings more beneficial.

Both DSM baselines run out of memory at batch size 64, whereas F-DSM can be trained at this batch size on both datasets. This allows larger batches to be processed directly, reducing the need for gradient accumulation and enabling more efficient training.

4.4. Inference Efficiency

As described in Sec. 3, during streaming inference, F-DSM first uses the padding prediction head to determine whether the model should output <p>. When <p> is selected, the projection and softmax over the large LLM vocabulary can be skipped.

In our experiments, F-DSM skips the large-vocabulary softmax on 74.70% of decoding steps on CSJ and 66.89% on LibriSpeech, as shown in Table 3. Thus, F-DSM avoids vocabulary computation on most decoding steps. F-DSM achieves the lowest RTF on both datasets (0.811 on CSJ and 0.759 on LibriSpeech), indicating a modest improvement in end-to-end inference speed. The gain is limited because the acoustic encoder and LLM decoder still need to be computed.

Table 4. Text-only language modeling after ASR adaptation. Renorm. PPL excludes special tokens and renormalizes probabilities over the original vocabulary. Special-token probability: summed raw-head probabilities of <p> or <w>.

Model	Raw PPL ↓	Renorm. PPL ↓	Special-token probability (%)
<i>CSJ</i>			
Base Qwen	4.401×10^2	—	—
DSM w/ <w>	1.858×10^6	3.293×10^5	51.77
DSM w/o <w>	3.113×10^6	2.407×10^5	77.97
F-DSM	2.388×10^5	2.388×10^5	0.002183
<i>LibriSpeech</i>			
Base Qwen	5.945×10^1	—	—
DSM w/ <w>	2.290×10^5	5.003×10^4	55.94
DSM w/o <w>	4.831×10^5	4.692×10^4	76.95
F-DSM	4.296×10^4	4.296×10^4	0.001579

4.5. Analysis of LLM Forgetting

Fine-tuning LLMs on downstream tasks can lead to catastrophic forgetting of previously learned capabilities [24]. We therefore evaluate text-only PPL to investigate how ASR adaptation affects the original language modeling capability of the LLM. The results are shown in Table 4.

F-DSM achieves the lowest raw PPL among the adapted models on both datasets, substantially mitigating the degradation observed with DSM. The DSM baselines also assign a large amount of probability mass to ASR-specific tokens during text-only evaluation. On CSJ, the probability mass assigned to special tokens is 51.77% for DSM w/ <w> and 77.97% for DSM w/o <w>, whereas it is only 0.002183% for F-DSM. A similar trend is observed on LibriSpeech.

Even after renormalizing the probability distribution over text tokens only, F-DSM achieves the lowest PPL. However, its advantage over both DSM baselines shrinks substantially, and all adapted models still have much higher PPL than the original LLM. These results suggest that gradients from <p> prediction may not be completely isolated from the LLM, even in F-DSM. Jointly providing audio and text embeddings to the LLM may also degrade its language modeling capability.

5. CONCLUSION

We proposed F-DSM, which separates the waiting probability for <p> from text-token prediction over the original LLM vocabulary. Experiments on CSJ and LibriSpeech showed that the word-start token <w> is not required for competitive recognition accuracy and that F-DSM outperforms the original DSM. F-DSM also greatly reduces GPU memory use while maintaining similar training throughput, and skips the large-vocabulary softmax on 74.70% of decoding steps on CSJ and 66.89% on LibriSpeech with modest gains in end-to-end inference speed. In addition, F-DSM reduces the degradation in text-only PPL observed with DSM, although a gap from the base LLM remains. Separating waiting from text prediction thus improves LLM-based streaming ASR. Future work will explore mechanisms to skip LLM inference itself, aiming to improve performance by further mitigating catastrophic forgetting and to reduce computational cost.

6. REFERENCES

- [1] Yanzhang He, Tara N. Sainath, Rohit Prabhavalkar, Ian McGraw, Raziq Alvarez, Ding Zhao, David Rybach, Anjuli Kannan, Yonghui Wu, Ruoming Pang, Qiao Liang, Deepti Bhatia, Yuan Shangguan, Bo Li, Golan Pundak, Khe Chai Sim, Tom Bagby, Shuo-yiin Chang, Kanishka Rao, and Alexander Gruenstein, “Streaming end-to-end speech recognition for mobile devices,” in *ICASSP 2019 - 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2019, pp. 6381–6385.
- [2] Vahid Noroozi, Somshubra Majumdar, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg, “Stateful conformer with cache-based inference for streaming automatic speech recognition,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 12041–12045.
- [3] Alex Graves, Santiago Fernández, Faustino Gomez, and Jürgen Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in *Proceedings of the 23rd International Conference on Machine Learning*, New York, NY, USA, 2006, ICML ’06, p. 369–376, Association for Computing Machinery.
- [4] Alex Graves, “Sequence transduction with recurrent neural networks,” 2012.
- [5] Humza Naveed, Asad Ullah Khan, Shi Qiu, Muhammad Saqib, Saeed Anwar, Muhammad Usman, Naveed Akhtar, Nick Barnes, and Ajmal Mian, “A comprehensive overview of large language models,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, Aug. 2025.
- [6] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao, “Large language models: A survey,” 2025.
- [7] Dong Zhang, Shimin Li, Xin Zhang, Jun Zhan, Pengyu Wang, Yaqian Zhou, and Xipeng Qiu, “SpeechGPT: Empowering large language models with intrinsic cross-modal conversational abilities,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, Houda Bouamor, Juan Pino, and Kalika Bali, Eds., Singapore, Dec. 2023, pp. 15757–15773, Association for Computational Linguistics.
- [8] Wenyi Yu, Changli Tang, Guangzhi Sun, Xianzhao Chen, Tian Tan, Wei Li, Lu Lu, Zejun Ma, and Chao Zhang, “Connecting speech encoder and large language model for asr,” in *ICASSP 2024 - 2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2024, pp. 12637–12641.
- [9] Xian Shi, Xiong Wang, Zhifang Guo, Yongqi Wang, Pei Zhang, Xinyu Zhang, Zishan Guo, Hongkun Hao, Yu Xi, Baosong Yang, Jin Xu, Jingren Zhou, and Junyang Lin, “Qwen3-asr technical report,” 2026.
- [10] Neil Zeghidour, Eugene Kharitonov, Manu Orsini, Václav Volhejn, Gabriel de Marmiesse, Edouard Grave, Patrick Pérez, Laurent Mazaré, and Alexandre Défossez, “Streaming sequence-to-sequence learning with delayed streams modeling,” 2025.
- [11] Mistral-AI, “Voxtral realtime,” 2026.
- [12] Ehsan Variani, David Rybach, Cyril Allauzen, and Michael Riley, “Hybrid autoregressive transducer (hat),” in *ICASSP 2020 - 2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 6139–6143.
- [13] Kikuo Maekawa, “Corpus of spontaneous Japanese: its design and evaluation,” in *ISCA/IEEE Workshop on Spontaneous Speech Processing and Recognition*, 2003, p. paper MM02.
- [14] Vassil Panayotov, Guoguo Chen, Daniel Povey, and Sanjeev Khudanpur, “Librispeech: An asr corpus based on public domain audio books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2015, pp. 5206–5210.
- [15] Max Bain, Jaesung Huh, Tengda Han, and Andrew Zisserman, “WhisperX: Time-accurate speech transcription of long-form audio,” in *Proc. INTERSPEECH 2023*, 2023, pp. 4489–4493.
- [16] Xie Chen, Zhong Meng, Sarangarajan Parthasarathy, and Jinyu Li, “Factorized neural transducer for efficient language model adaptation,” in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 8132–8136.
- [17] Duc Le, Frank Seide, Yuhao Wang, Yang Li, Kjell Schubert, Ozlem Kalinli, and Michael L. Seltzer, “Factorized blank thresholding for improved runtime efficiency of neural transducers,” in *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023, pp. 1–5.
- [18] Takafumi Moriya, Takanori Ashihara, Masato Mimura, Hiroshi Sato, Kohei Matsuura, Ryo Masumura, and Taichi Asami, “Boosting Hybrid Autoregressive Transducer-based ASR with Internal Acoustic Model Training and Dual Blank Thresholding,” in *Interspeech 2024*, 2024, pp. 3465–3469.
- [19] Qwen Team, “Qwen3.5: Towards native multimodal agents,” February 2026.
- [20] Dima Rekeshe, Nithin Rao Koluguri, Samuel Krizan, Somshubra Majumdar, Vahid Noroozi, He Huang, Oleksii Hrinchuk, Krishna Puvvada, Ankur Kumar, Jagadeesh Balam, and Boris Ginsburg, “Fast conformer with linearly scalable attention for efficient speech recognition,” in *2023 IEEE Automatic Speech Recognition and Understanding Workshop (ASRU)*, 2023, pp. 1–8.
- [21] NVIDIA, “Nemotron 3.5 ASR,” <https://huggingface.co/nvidia/nemotron-3.5-asr-streaming-0.6b>, 2026, Hugging Face model card, accessed Sep. 24, 2026.
- [22] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [23] Ilya Loshchilov and Frank Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations*, 2019.
- [24] Yun Luo, Zhen Yang, Fandong Meng, Yafu Li, Jie Zhou, and Yue Zhang, “An empirical study of catastrophic forgetting in large language models during continual fine-tuning,” *IEEE Transactions on Audio, Speech and Language Processing*, vol. 33, pp. 3776–3786, 2025.